

C parser for grammar

After taking user input, the input must be parsed so that we can decide what the user wants the shell to do. You will be implementing a parser for the below Context Free Grammar

shell_cmd -> cmd_group ((& | &&) cmd_group)* &?

cmd_group -> atomic (\ | atomic)*

atomic -> name (name | input | output)*

input -> < name | <name

output -> > name | >name | >> name | >>name

name -> r"^[^&><;]+"

A simple introduction to Context Free Grammars and an example parser for a simpler grammar will be provided in the tutorial.

Parsers for such programs usually create something called an Abstract Syntax Tree. However, you are not required to parse the input into an AST. You can use any structure that you find the most convenient!

explain in detail with examples and suggest data structures to implement these efficiently in C

here's a practical way to turn that grammar into a fast, no-nonsense C parser. i'll walk through:

- what the grammar means (with examples)
- a minimal-but-solid lexer design (handles `<name`, `>>file`, etc.)
- lightweight C data structures for the parse result
- a recursive-descent parser that matches your grammar
- examples that show what the parser will produce
- tips for robustness & performance

1) reading the grammar (quick intuition)

pgsql

```
shell_cmd -> cmd_group ((& | &&) cmd_group)* &?  
cmd_group -> atomic ( '|' atomic )*  
atomic   -> name (name | input | output)*  
input    -> '<' name | '<name'  
output   -> '>' name | '>name' | '>>' name | '>>name'  
name     -> r"^[&><;]+"
```

- `shell_cmd` is one or more `cmd_group` s separated by `&` or `&&` . optionally a final trailing `&` .

examples:

- `sleep 1 &` → one group, background `&?` present
- `make && make install` → two groups, joined by `&&`
- `cmd1 & cmd2 & cmd3 &` → three groups each separated by `&` , and also a trailing `&`
- `cmd_group` is a pipeline: one or more `atomic` s joined by `|` .
example: `grep foo file | sort | uniq -c`
- `atomic` is a command name followed by zero or more **arguments (name)** or **redirections**.

examples:

- `cat <in.txt >out.txt`
- `echo hi >>log.txt`
- `sort -n -r <in >out`
- `input / output` allow both spaced and compact forms:
 - spaced: `< name , >> name`
 - compact: `<name , >>name`
- `name` is any run of characters that are **not** one of `| & > < ;` . (so spaces split names; those punctuation chars end names)

2) tokenization (lexer)

Goal: produce a clean token stream for the parser, while accepting both spaced and compact redirections.

Token types

c

```

typedef enum {
    T_EOF = 0,
    T_NAME,      // text for command or argument (no | & > <;)
    T_PIPE,      // |
    T_AMP,        // &
    T_AND_AND,    // &&
    T_LT,         // <
    T_GT,         // >
    T_GT_GT       // >>
} TokenType;

typedef struct {
    TokenType type;
    const char* lexeme; // pointer into original buffer (optional copy)
    int len;           // length of lexeme (for NAME mainly)
    int pos;           // byte offset (for error messages)
} Token;

```

Handling `<name` , `>name` , `>>name`

When you see `<` , `>` , or `>>` , **emit the operator token** first. If the next characters immediately form a `name` with no whitespace, emit a `T_NAME` next. This way the grammar's compact forms still parse as `<` then `name` .

Pseudocode (core idea):

```

c

skip_spaces();
if (peek() == '&') {
    if (peek2() == '&') { emit(T_AND_AND); advance(2); }
    else { emit(T_AMP); advance(1); }
} else if (peek() == '|') {
    emit(T_PIPE); advance(1);
} else if (peek() == '<') {
    emit(T_LT); advance(1);
    if (next_is_name_char()) emit_name(); // handles <name
} else if (peek() == '>') {
    if (peek2() == '>') { emit(T_GT_GT); advance(2); }
    else { emit(T_GT); advance(1); }
    if (next_is_name_char()) emit_name(); // handles >name / >>name
} else if (is_name_char(peek())) {

```

```

    emit_name();                // NAME: run until special or space
} else if (peek() == ';') {
    // not in grammar; treat as error or end-of-command if you choose to extend
    error("unexpected ';'");
} else if (peek() == '\0') {
    emit(T_EOF);
} else {
    error("bad character");
}

```

Where `is_name_char(c)` is true iff `c` is **not** whitespace and not one of `|&><;` .

This scanner is $O(n)$ and simple.

3) parse result data structures (no need for a full AST)

Efficient & convenient structs that mirror your grammar and are easy to execute:

```

c

typedef struct {
    char *in_path;    // nullable
    char *out_path;   // nullable
    int  append;      // 0: truncate (>), 1: append (>>)
    char **argv;      // NULL-terminated: ["cmd", "arg1", "arg2", NULL]
    int  argc;        // count not including NULL
} Atomic;

typedef struct {
    Atomic *atoms;    // pipeline atoms: atoms[0] | atoms[1] | ...
    int  natoms;
} CmdGroup;

typedef enum { SEP_AMP, SEP_AND_AND } SepType;

typedef struct {
    CmdGroup *groups; // groups[0] (sep[0]) groups[1] (sep[1]) groups[2] ...
    SepType *seps;    // number of separators == ngroups - 1
    int  ngroups;
    int  trailing_amp; // 1 if final &? present
} ShellCmd;

```

Memory strategy:

- Use simple `realloc`-grown arrays during parsing (amortized $O(1)$ appends).
- Copy `name` lexemes into owned, NUL-terminated strings (so you can free later).
- If you care about peak performance, switch to an arena allocator, but `realloc` is totally fine here.

4) recursive-descent parser (straight from the grammar)

Function signatures:

```
c

// global-ish parser state:
typedef struct { Token *toks; int i; int n; } P;

ShellCmd parse_shell_cmd(P *p);
CmdGroup parse_cmd_group(P *p);
Atomic parse_atomic(P *p);

// token helpers:
Token *peek(P *p);
Token *consume(P *p, TokenType t, const char *expect_msg);
int match(P *p, TokenType t); // if matches, consume & return 1 else 0
```

parse_atomic

```
pgsql

atomic -> name (name | input | output)*
input -> '<' name
output -> '>' name | '>>' name
```

Implementation outline:

```
c

Atomic parse_atomic(P *p) {
    Atomic a = {0};
    dyn_argv_init(&a); // push strings; we'll add the final NULL later
```

```

// command name
Token *t = consume(p, T_NAME, "expected command name");
dyn_argv_push(&a, strdup(t->lexeme, t->len));

// zero or more: NAME | input | output
for (;;) {
    if (match(p, T_NAME)) {
        Token *arg = &p->toks[p->i-1];
        dyn_argv_push(&a, strdup(arg->lexeme, arg->len));
    } else if (match(p, T_LT)) {
        Token *nm = consume(p, T_NAME, "expected filename after '<'");
        if (a.in_path) error_at(nm, "duplicate input redirection");
        a.in_path = strdup(nm->lexeme, nm->len);
    } else if (match(p, T_GT_GT)) {
        Token *nm = consume(p, T_NAME, "expected filename after '>>'");
        if (a.out_path) error_at(nm, "duplicate output redirection");
        a.out_path = strdup(nm->lexeme, nm->len);
        a.append = 1;
    } else if (match(p, T_GT)) {
        Token *nm = consume(p, T_NAME, "expected filename after '>'");
        if (a.out_path) error_at(nm, "duplicate output redirection");
        a.out_path = strdup(nm->lexeme, nm->len);
        a.append = 0;
    } else {
        break;
    }
}

dyn_argv_null_terminate(&a);
return a;
}

```

parse_cmd_group

sql

cmd_group -> atomic ('|' atomic)*

c

```

CmdGroup parse_cmd_group(P *p) {
    CmdGroup g = {0};
    dyn_atoms_push(&g, parse_atomic(p));
    while (match(p, T_PIPE)) {
        dyn_atoms_push(&g, parse_atomic(p));
    }
    return g;
}

```

parse_shell_cmd

rust

```
shell_cmd -> cmd_group ((& | &&) cmd_group)* &?
```

c

```

ShellCmd parse_shell_cmd(P *p) {
    ShellCmd sc = {0};
    dyn_groups_push(&sc, parse_cmd_group(p));
    for (;;) {
        if (match(p, T_AND_AND)) {
            dyn_seps_push(&sc, SEP_AND_AND);
            dyn_groups_push(&sc, parse_cmd_group(p));
        } else if (match(p, T_AMP)) {
            // could be a separator between groups OR the trailing &?.
            // disambiguate by peeking: if next starts a cmd_group (i.e., NAME),
            // treat it as a separator; otherwise it's the trailing &.
            if (peek(p)->type == T_NAME) {
                dyn_seps_push(&sc, SEP_AMP);
                dyn_groups_push(&sc, parse_cmd_group(p));
            } else {
                sc.trailing_amp = 1;
                break;
            }
        } else {
            break;
        }
    }
    consume(p, T_EOF, "unexpected tokens after command"); // or allow partial
}

```

```
    return sc;
}
```

Note: The “separator vs trailing &” rule above is a simple, deterministic lookahead: a new group **must** start with a `NAME` .

5) end-to-end examples

Example A

```
bash

echo hello >out.txt &
```

Parse result (conceptually):

```
pgsql

ShellCmd {
  groups = [
    CmdGroup {
      atoms = [
        Atomic { argv=["echo","hello"], in=NULL, out="out.txt", append=0 }
      ]
    }
  ],
  seps=[], trailing_amp=1
}
```

Example B

```
bash

cat<in.txt | grep -i error | sort >>log && notify-send done
```

Tokenization includes `<` then NAME `in.txt` (no space) and `>>` then NAME `log` (no space).

Result:

```
pgsql
```



```

ShellCmd {
  groups = [
    CmdGroup {
      atoms = [
        { argv=["cat"],   in="in.txt", out=NULL },
        { argv=["grep","-i","error"], in=NULL, out=NULL },
        { argv=["sort"],   out="log", append=1 }
      ]
    },
    CmdGroup {
      atoms = [
        { argv=["notify-send","done"] }
      ]
    }
  ],
  seps=[SEP_AND_AND],
  trailing_amp=0
}

```

Example C

bash

build & test && deploy &

Result:

vbnet

```

groups: [ ["build"], ["test"], ["deploy"] ]
seps:   [ SEP_AMP, SEP_AND_AND ]
trailing_amp: 1

```

6) execution notes (if you later hook this to a runner)

- For each `CmdGroup`, create N pipes for N-1 `Atomic` s; set up stdin/stdout per `in_path` / `out_path` (`O_CLOEXEC`, `O_TRUNC` vs `O_APPEND`).
- `SEP_AND_AND` : only run next group if previous **succeeded** (exit status 0).

- `SEP_AMP` : run previous group, don't wait (background). Whether to start the **next** group immediately or queue is up to your shell semantics; typical shells run it and return to prompt.
 - `trailing_amp` : after the last group is launched, return to prompt without waiting.
-

7) practical C scaffolding (mini library bits)

simple growing arrays

```
c

#define GROW_CAP(cap) ((cap) < 8 ? 8 : (cap) * 2)

#define VEC_PUSH(vec, n, cap, value) do { \
    if ((n) == (cap)) { \
        (cap) = GROW_CAP(cap); \
        (vec) = realloc((vec), (cap) * sizeof(*(vec))); \
    } \
    (vec)[(n)++] = (value); \
} while (0)
```

Use it to implement `dyn_argv_push`, `dyn_atoms_push`, etc.

argv helpers

```
c

static void dyn_argv_init(Atomic *a) { a->argv = NULL; a->argc = 0; }
static void dyn_argv_push(Atomic *a, char *s) {
    static int cap = 0; // better: store per-Atomic
    // (for real code: store cap in Atomic; shown short here)
}
static void dyn_argv_null_terminate(Atomic *a) {
    a->argv = realloc(a->argv, (a->argc + 1) * sizeof(char*));
    a->argv[a->argc] = NULL;
}
```

(For production, keep `argv_cap` inside `Atomic`.)

errors with position

Keep `pos` (byte index) per token; on error, print a caret-line by slicing the original input.

8) corner cases & policy decisions

- **Multiple input (`<`) or output redirections:** error out or “last one wins”. The code above errors on duplicates.
 - **Empty pipeline stage:** grammar forbids `|` without an `atomic` on each side—your parser will throw a clear error.
 - **Weird chars inside names:** allowed unless they’re one of `|&><;` or whitespace. So `file:name` is fine, `a;b` is not.
 - **`&&&` typo:** the lexer will emit `&&` then `&`, which the parser treats as a separator and maybe trailing `&`—you can add a validation pass to reject this if you like.
 - `;`: not in your grammar—reject it, or extend the grammar later.
-

9) complexity & efficiency

- Lexer: $O(n)$ single pass; minimal backtracking (only to decide `&&` and `>>`).
 - Parser: each token consumed once $\rightarrow O(n)$.
 - Memory: proportional to number of tokens + total characters copied for names.
-

if you want, i can package this into a single `.c` file with the full lexer + parser skeleton, a tiny driver that prints the parsed structure, and a few unit tests—just say the word and i’ll drop it in.

After taking user input, the input must be parsed so that we can decide what the user wants the shell to do. You will be implementing a parser for the below Context Free Grammar

```
shell_cmd -> cmd_group ((& | &&) cmd_group)* &?
cmd_group -> atomic (\| atomic)*
atomic -> name (name | input | output)*
input -> < name | <name
```

```
output -> > name | >name | >> name | >>name
name -> r"^[^<>|&]+"
```

how do i implement this in C, very simply?

Got it 👍 — you don't need a full compiler-style parser here. Since your grammar is **pretty regular** (no ambiguity, no recursion except lists), you can write a **hand-rolled parser in C** with just:

1. **Tokenize** the input string (split into tokens like `NAME`, `<`, `>`, `>>`, `|`, `&`, `&&`).
2. **Recursive-descent parser** functions that mirror the grammar.
3. **Simple structs** to hold results (arrays of command groups, each with argv, input/output, etc.).

I'll give you a **minimal but working skeleton**.

Step 1: Define tokens

```
c

typedef enum {
    T_EOF = 0,
    T_NAME,
    T_PIPE,    // |
    T_AMP,     // &
    T_AND_AND, // &&
    T_LT,      // <
    T_GT,      // >
    T_GT_GT    // >>
} TokenType;

typedef struct {
    TokenType type;
    char *text; // only for T_NAME
} Token;
```

Step 2: Tokenizer (lexer)

This reads the string and produces a list of tokens.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAXTOK 256

Token tokens[MAXTOK];
int ntok = 0;

void add_token(TokenType type, const char *start, int len) {
    tokens[ntok].type = type;
    if (type == T_NAME) {
        tokens[ntok].text = strdup(start, len);
    } else {
        tokens[ntok].text = NULL;
    }
    ntok++;
}

void tokenize(const char *s) {
    const char *p = s;
    while (*p) {
        if (isspace(*p)) { p++; continue; }
        if (*p == '&') {
            if (*(p+1) == '&') { add_token(T_AND_AND, NULL, 0); p+=2; }
            else { add_token(T_AMP, NULL, 0); p++; }
        } else if (*p == '|') {
            add_token(T_PIPE, NULL, 0); p++;
        } else if (*p == '<') {
            add_token(T_LT, NULL, 0); p++;
        } else if (*p == '>') {
            if (*(p+1) == '>') { add_token(T_GT_GT, NULL, 0); p+=2; }
            else { add_token(T_GT, NULL, 0); p++; }
        } else {
            // NAME: run until space or special char
            const char *start = p;
            while (*p && !isspace(*p) && !strchr("&|><", *p)) p++;
            add_token(T_NAME, start, p-start);
        }
    }
}

```

```
}  
add_token(T_EOF, NULL, 0);  
}
```

Step 3: Data structures for parsed result

```
c  
  
typedef struct {  
    char **argv;    // arguments  
    char *infile;   // < file  
    char *outfile;  // > file  
    int append;     // >> ?  
} Atomic;  
  
typedef struct {  
    Atomic *atoms;  // pipeline: atom | atom | atom  
    int natoms;  
} CmdGroup;  
  
typedef struct {  
    CmdGroup *groups; // multiple groups separated by & / &&  
    int ngroups;  
    int trailing_amp;  
} ShellCmd;
```

Step 4: Very simple parser

We just need recursive functions for each grammar rule.

```
c  
  
int pos = 0;  
Token *peek() { return &tokens[pos]; }  
Token *get() { return &tokens[pos++]; }  
  
Atomic parse_atomic() {  
    Atomic a = {0};  
    a.argv = malloc(sizeof(char*)*32);
```

```

int argc = 0;

// must start with NAME
if (peek()->type != T_NAME) {
    fprintf(stderr, "expected command name\n"); exit(1);
}
a.argv[argc++] = strdup(get()->text);

// loop over [name | input | output]*
while (1) {
    if (peek()->type == T_NAME) {
        a.argv[argc++] = strdup(get()->text);
    } else if (peek()->type == T_LT) {
        get(); // consume <
        if (peek()->type != T_NAME) { fprintf(stderr, "expected infile\n"); exit(1); }
        a.infile = strdup(get()->text);
    } else if (peek()->type == T_GT || peek()->type == T_GT_GT) {
        int append = (peek()->type == T_GT_GT);
        get();
        if (peek()->type != T_NAME) { fprintf(stderr, "expected outfile\n"); exit(1); }
        a.outfile = strdup(get()->text);
        a.append = append;
    } else break;
}

a.argv[argc] = NULL;
return a;
}

CmdGroup parse_cmd_group() {
    CmdGroup g = {0};
    g.atoms = malloc(sizeof(Atomic)*32);
    g.natoms = 0;

    g.atoms[g.natoms++] = parse_atomic();
    while (peek()->type == T_PIPE) {
        get(); // consume |
        g.atoms[g.natoms++] = parse_atomic();
    }
    return g;
}

```

```

ShellCmd parse_shell_cmd() {
    ShellCmd sc = {0};
    sc.groups = malloc(sizeof(CmdGroup)*32);
    sc.ngroups = 0;

    sc.groups[sc.ngroups++] = parse_cmd_group();
    while (peek()->type == T_AMP || peek()->type == T_AND_AND) {
        TokenType sep = get()->type;
        if (peek()->type == T_NAME) {
            sc.groups[sc.ngroups++] = parse_cmd_group();
        } else {
            if (sep == T_AMP) sc.trailing_amp = 1;
            break;
        }
    }
    return sc;
}

```

Step 5: Test it

c

```

int main() {
    char line[256];
    printf("shell> ");
    fgets(line, sizeof(line), stdin);

    tokenize(line);
    ShellCmd cmd = parse_shell_cmd();

    printf("Parsed %d group(s)\n", cmd.ngroups);
    for (int g=0; g<cmd.ngroups; g++) {
        for (int a=0; a<cmd.groups[g].natoms; a++) {
            Atomic *at = &cmd.groups[g].atoms[a];
            printf("  cmd: ");
            for (int i=0; at->argv[i]; i++)
                printf("%s ", at->argv[i]);
            if (at->infile) printf("< %s ", at->infile);
            if (at->outfile) printf("%s %s ", at->append?">":">", at->outfile);
            printf("\n");
        }
    }
}

```



```
}  
if (cmd.trailing_amp) printf("(background)\n");  
}
```

Example runs

csharp

```
shell> echo hello >out.txt &  
Parsed 1 group(s)  
cmd: echo hello > out.txt  
(background)
```

bash

```
shell> cat <in.txt | grep foo >>log  
Parsed 1 group(s)  
cmd: cat < in.txt  
cmd: grep foo >> log
```

go

```
shell> make && make install  
Parsed 2 group(s)  
cmd: make  
cmd: make install
```

✓ This is a **very simple recursive-descent parser in C**, closely following your grammar.

It gives you clean data structures (`ShellCmd` → groups → atoms) you can later execute with `fork/exec/dup2`.

Do you want me to extend this so the parser also **executes the commands** (like a minimal working shell), or keep it **only for parsing**?

```
typedef enum {
    T_EOF = 0,
    T_NAME,
    T_PIPE,
    T_AMP,
    T_AND_AND,
    T_LT,
    T_GT,
    T_GT_GT
} TokenType;
```

```
typedef struct {
    TokenType type;
    char* text;
} Token;
```

```
typedef struct {
    char **argv;
    char *infile;
    char *outfile;
    int append;
} Atomic;
```

```
typedef struct {
    Atomic *atoms;
    int natoms;
} CmdGroup;
```

```
typedef struct {
    CmdGroup *groups;
    int ngroups;
    int trailing_amp;
} ShellCmd;
```

```
void add_token(TokenType type, const char *start, int
len, Token* tokens, int* ntok);
void tokenize(const char *s, Token* tokens;
CmdGroup parse_cmd_group();
ShellCmd parse_shell_cmd();
Atomic parse_atomic();
Token* peek();
```

```
Token* get();
```

why does this error out?

ah, i see what's happening 😊 — the snippet you pasted has a few **C declaration mistakes**.

let me go line by line:

1. this line is wrong:

c

```
void tokenize(const char *s), Token* tokens;
```

- in C you **cannot declare two things separated by a comma like that** unless they share the same type.
- here you probably meant:

c

```
void tokenize(const char *s, Token *tokens, int *ntok);
```

so `tokenize` knows where to put tokens and how many.

2. these forward declarations:

c

```
CmdGroup parse_cmd_group();  
ShellCmd parse_shell_cmd();  
Atomic parse_atomic();  
Token* peek();  
Token* get();
```

they are fine as function prototypes. but if you later define them, make sure the signatures **match exactly**.

example:

c

```
Atomic parse_atomic(void); // not just parse_atomic()
```

if you leave empty `()`, old-style C means "any arguments" which can cause warnings or errors with `-Wall -Werror`.

so better:

c

```
CmdGroup parse_cmd_group(void);  
ShellCmd parse_shell_cmd(void);  
Atomic parse_atomic(void);  
Token* peek(void);  
Token* get(void);
```

3. this one also has a small bug:

c

```
void add_token(TokenType type, const char *start, int len, Token* tokens, int* ntok);
```

that's okay *if you plan to pass the token array in*. but in the earlier code you showed me, `tokens` and `ntok` were global.

so either:

- keep globals:

c

```
void add_token(TokenType type, const char *start, int len);
```

csharp

and inside `add_token` just update the `global` `tokens[]` and `ntok`.

- or keep them `explicit` (like your signature), but then you must call it `with` those arguments.

✓ ****fixed** prototypes:

```c

void add\_token(TokenType type, const char \*start, int len, Token \*tokens, int \*ntok);

void tokenize(const char \*s, Token \*tokens, int \*ntok);

CmdGroup parse\_cmd\_group(void);

ShellCmd parse\_shell\_cmd(void);

Atomic parse\_atomic(void);

Token\* peek(void);

Token\* get(void);

👉 so the error you're seeing is from `void tokenize(const char *s), Token* tokens;` — that's invalid C.

do you want me to rewrite a **compilable minimal file** ( `parser.c` ) with the corrected headers + a toy `main` that tokenizes and parses one line?

```
#include "shell.h"
```

```
#define MAXTOK 256
```

```
Token tokens[MAXTOK];
```

```
int ntok = 0;
```

```
int main()
```

```
{
```

```
 char user[1024];
```

```
 char systemName[1024];
```

```
 //////////// LLM Generated Code Begins ////////////
```

```
 strcpy(user, getenv("USER"));
```

```
 if (gethostname(systemName, 1024) != 0)
```

```
 {
```

```
 printf("Environment Variable (HOSTNAME) error.");
```

```

 exit(1);
 }

 //printf("USER: %s\n", user);
 //printf("SYSTEMNAME: %s\n", systemName);

 //////////// LLM Generated Code Ends ////////////

 while (1)
 {
 char* dir_name = make_init_dir_name();

 char* display = make_init_display(user,
systemName, dir_name);

 printf("%s", display);
 free(display);

 char* input = malloc(1024);
 scanf("%1023[^\n]", input);

 scanf("%*[^\\n]");
 scanf("%*c");

 tokenize(input, &tokens);
 ShellCmd cmd = parse_shell_cmd();

 printf("Parsed %d group(s)\n", cmd.ngroups);
 for (int g=0; g<cmd.ngroups; g++) {
 for (int a=0; a<cmd.groups[g].natoms; a++) {
 Atomic *at = &cmd.groups[g].atoms[a];
 printf(" cmd: ");

 for (int i=0; at->argv[i]; i++)
 printf("%s ", at->argv[i]);

 if (at->infile) printf("< %s ", at->infile);

 if (at->outfile) printf("%s %s ", at-
>append?">>":">", at->outfile);

 printf("\n");
 }
 }
 }
}

```

```

 }
}

free(input);
free(dir_name);
}

return 0;
}

```

would this be correct?

this is **very close**, but a few small corrections are needed to make it compile and behave correctly.

## 1. your `tokenize` call

you wrote:

```

c

tokenize(input, &tokens);

```

but in your header you had declared:

```

c

void tokenize(const char *s, Token *tokens, int *ntok);

```

so the call should be:

```

c

ntok = 0; // reset before each scan
tokenize(input, tokens, &ntok);

```

## 2. `parse_shell_cmd` and globals

your `parse_shell_cmd()` currently takes no arguments.

but inside, it uses `tokens[]` and a global `pos`. that's fine, but then **before calling `parse_shell_cmd` you must reset parser state:**

```
c

pos = 0; // reset token index
ShellCmd cmd = parse_shell_cmd();
```

otherwise it will keep reading from the previous command.

---

### 3. `scanf` input reading

this part:

```
c

scanf("%1023[^\n]", input);
scanf("%*[^\\n]");
scanf("%*c");
```

is a bit fragile. if the user just presses enter, the first `scanf` will fail.

simpler: use `fgets` :

```
c

if (!fgets(input, 1024, stdin)) break; // EOF
input[strcspn(input, "\\n")] = '\\0'; // strip newline
```

---

### 4. freeing strings

you're freeing `input` and `dir_name`, good.

but `tokenize` allocates `text` for `T_NAME` with `strdup`. you'll leak memory unless you `free` tokens after parsing.

so after processing one loop:

```
c

for (int i=0; i<ntok; i++) {
 free(tokens[i].text);
```



```
}
```

## 5. `make_init_dir_name` / `make_init_display`

you call these, so i assume you implemented them elsewhere.

just be sure they return malloc'd strings, since you `free` them.

✓ corrected loop skeleton:

```
c
```

```
while (1) {
 char* dir_name = make_init_dir_name();
 char* display = make_init_display(user, systemName, dir_name);

 printf("%s", display);
 free(display);

 char* input = malloc(1024);
 if (!fgets(input, 1024, stdin)) break;
 input[strcspn(input, "\n")] = '\0';

 ntok = 0;
 tokenize(input, tokens, &ntok);

 pos = 0;
 ShellCmd cmd = parse_shell_cmd();

 printf("Parsed %d group(s)\n", cmd.ngroups);
 for (int g=0; g<cmd.ngroups; g++) {
 for (int a=0; a<cmd.groups[g].natoms; a++) {
 Atomic *at = &cmd.groups[g].atoms[a];
 printf(" cmd: ");
 for (int i=0; at->argv[i]; i++)
 printf("%s ", at->argv[i]);
 if (at->infile) printf("< %s ", at->infile);
 if (at->outfile) printf("%s %s ", at->append?">":">", at->outfile);
 printf("\n");
 }
 }
```

```
}

for (int i=0; i<ntok; i++) free(tokens[i].text);
free(input);
free(dir_name);
}
```

---

so:

- ✓ reset `ntok` and `pos` before parsing
  - ✓ call `tokenize(input, tokens, &ntok)`
  - ✓ prefer `fgets` over tricky `scanf`
  - ✓ free token texts each loop
- 

do you want me to also give you a **tiny** `shell.h` with correct prototypes so this compiles cleanly?